Android

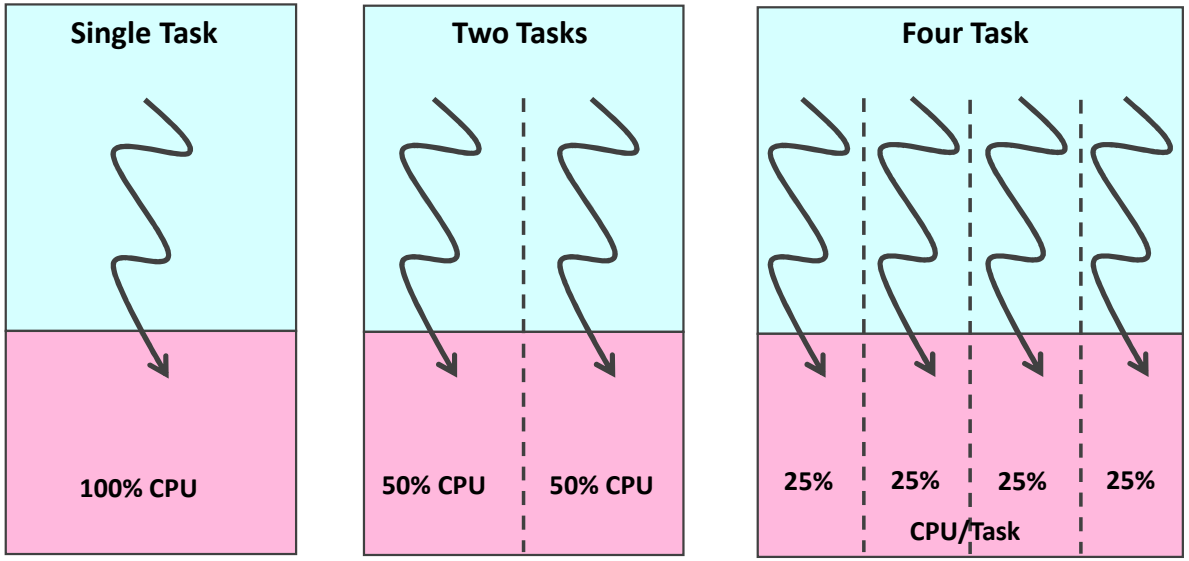
Scheduling

Scheduling de Processus Linux

- Sous Linux, les threads sont l'unité d'exécution fondamentale
  - Scheduling sous Linux concerne les threads et non pas les processus
- Scheduling alloue le temps d'exécution pour les threads sur un processeur
- Scheduler décide
  - Quel thread doit être exécuté
  - Combien de temps le thread est autorisé à s'exécuter avant de choisir un autre
- Dans Android, l'exécution des threads est planifiée en utilisant le scheduler du noyau Linux
  - **Completely Fair Scheduler (CFS)**

Completely Fair Scheduler

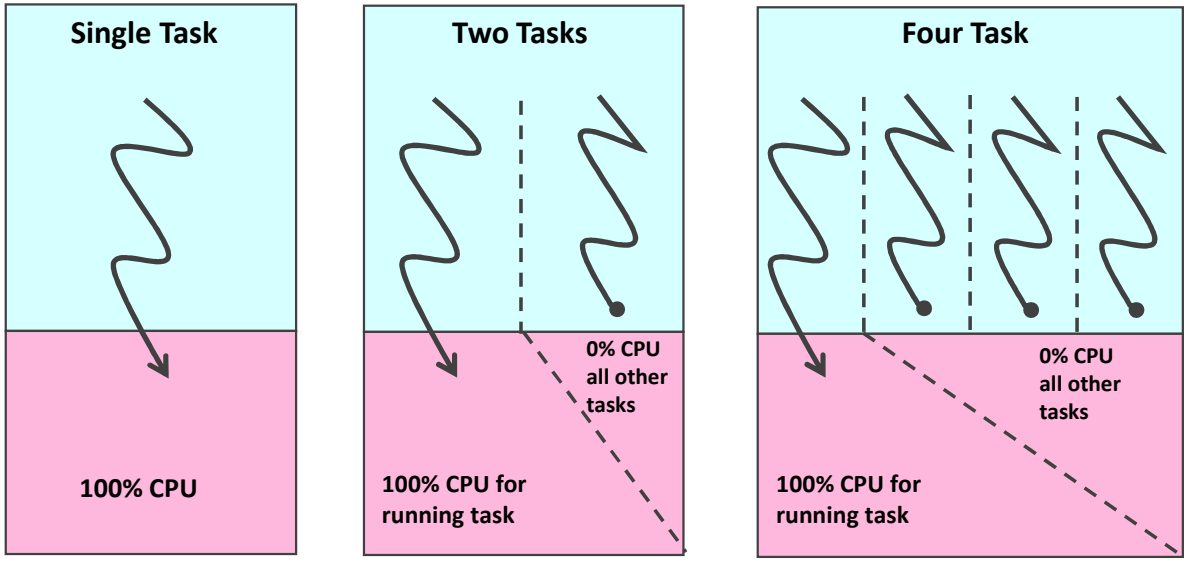
- Principe
  - Modélise un **processeur multitâche idéal**



Ideal Precise Multi-tasking CPU – Each task runs in parallel and consumes equal CPU share

Completely Fair Scheduler

- Principe
  - Modélise un **processeur multitâche idéal**



Actual CPU – While one task uses the CPU, every other task waits

Completely Fair Scheduler

- Exécute les tâches avec le **besoin le plus grave** (**gravest need**)

- Essaie de garantir l'equitabilité dans l'utilisation de l'UC

Completely Fair Scheduler

- Poids (Weight)
  - **Weight(nice) ≈ 1024/1.25^(nice)**
  - Les poids des valeurs nice sont pré-calculées pour éviter la **surcharge d'exécution** (**runtime overhead**)

```
static const int prio_to_weight[40] = {
 /* -20 */     88761,     71755,     56483,     46273,     36291,
 /* -15 */     29154,     23254,     18705,     14949,     11916,
 /* -10 */      9548,      7620,      6100,      4904,      3906,
 /*  -5 */      3121,      2501,      1991,      1586,      1277,
 /*   0 */      1024,       820,       655,       526,       423,
 /*   5 */       335,       272,       215,       172,       137,
 /*  10 */       110,        87,        70,        56,        45,
 /*  15 */        36,        29,        23,        18,        15,
};
```

Completely Fair Scheduler

- Poids (Weight)
  - Exemple
    - Pour nice = -1 (plus haute priorité), weight = 1277
    - Pour nice = 1 (plus faible priorité), weight = 820
    - Pour nice = 0 (priorité normale), weight = 1024

Completely Fair Scheduler

- Latence ciblée (Targeted Latency)
  - Période dans laquelle toutes les tâches (dans l'état runnable) sont exécutées au moins une fois

Completely Fair Scheduler

- **Quantum** (**Time slice**) pour une tâche i

$$\frac{TL * W_i}{\sum_{j=1}^{n} W_j}$$

où

- **TL** : Latence ciblée
- $\mathbf{W_i}$: le poids de la tâche i
- ***n*** : le nombre de tâches dans l'état runnable
- $\sum_{j=1}^{n} W_j$: la somme des poids des ***n*** tâches

Completely Fair Scheduler

- Quantum (time slice) pour une tâche i
  - Exemple
    - Latence ciblée = 100ms
    - T1 avec une valeur nice = 0; weight = 1024
    - T2 avec une valeur nice = 5; weight = 335
    - Quantum de T1 = 100*1024/(1024+335) ≈ 75ms
    - Quantum de T2 = 100* 335/(1024+335) ≈ 25ms

Completely Fair Scheduler

- vruntime
  - Le processus avec le **plus petit temps d'exécution virtuel** (**Virtual Run Time**) est sélectionné par le scheduler
  - Valeurs nice élevées doivent résulter en moins de temps CPU alloué à un processus
    - vruntime ne peut pas être égal au temps d'exécution réel (real runtime)

Completely Fair Scheduler

- vruntime
  - Exemple
    - Soit un processus avec une durée d'exécution de 200ms
    - Valeur nice = 0 → vruntime = 200ms
    - Valeur nice < 0 → vruntime < 200ms
    - Valeur nice > 0 → vruntime > 200ms

Completely Fair Scheduler

- vruntime
  - Soit **t** la durée de temps passée par un processus sur l'UC
  - vruntime est incrémenté par

$$\frac{t * W_0}{W_i}$$

Où

- W_0 : le poids de la valeur nice **0**
- W_i : le poids de la valeur nice ***i***

- W_0/W_i est appelée **facteur de décroissance** (**decay factor**)

Completely Fair Scheduler

- vruntime
  - Exemple 1
    - T1 avec nice value = 0 → weight = 1024
    - T2 avec nice value = 5 → weight = 335
    - T1 s'exécute pour 1 ns → vruntime += ?
    - T2 s'exécute pour 1 ns → vruntime += ?

Completely Fair Scheduler

- vruntime
  - Exemple 1
    - T1 avec nice value = 0 → weight = 1024
    - T2 avec nice value = 5 → weight = 335
    - T1 s'exécute pour 1 ns → vruntime += 1 (= 1*1024/1024)
    - T2 s'exécute pour 1 ns → vruntime += 3.05 (= 1*1024/335)

Completely Fair Scheduler

- vruntime
  - Exemple 2
    - T1 avec nice value = 0 → weight = 1024
    - T2 avec nice value = 0 → weight = 1024
    - vruntime1 = vruntime2 = 0
    - T1 s'exécute pour 200ms → vruntime1 = ?
    - T2 s'exécute pour 100ms → vruntime2 = ?
    - Quelle tâche sera sélectionnée en premier par le scheduler ?

Completely Fair Scheduler

- vruntime
  - Exemple 2
    - T1 avec nice value = 0 → weight = 1024
    - T2 avec nice value = 0 → weight = 1024
    - vruntime1 = vruntime2 = 0
    - T1 s'exécute pour 200ms → vruntime1 = 200ms
    - T2 s'exécute pour 100ms → vruntime2 = 100ms
    - T2 sera sélectionnée avant T1 par le scheduler

Completely Fair Scheduler

- vruntime
  - Exemple 3
    - T1 avec nice value = 0 → weight = 1024 → Decay factor = ?
    - T2 avec nice value = 5 → weight = 335 → Decay factor = ?
    - vruntime1 = vruntime2 = 0
    - T1 s'exécute pour 200ms → vruntime1 = ?
    - T2 s'exécute pour 100ms → vruntime2 = ?
    - Quelle tâche sera sélectionnée en premier par le scheduler ?
    - Supposant que T1 s'exécute pour encore 200ms
      - vruntime1 = ?
      - Quelle tâche sera sélectionnée maintenant par le scheduler ?

Completely Fair Scheduler

- vruntime
  - Exemple 3
    - T1 avec nice value = 0 → weight = 1024 → Decay factor = 1
    - T2 avec nice value = 5 → weight = 335 → Decay factor = 3.05
    - vruntime1 = vruntime2 = 0
    - T1 s'exécute pour 200ms → vruntime1 = 200ms
    - T2 s'exécute pour 100ms → vruntime2 = 305ms
    - T1 sera sélectionnée avant T2
    - Supposant que T1 s'exécute pour encore 200ms
      - vruntime1 = 400ms
      - T2 sera sélectionnée avant T1

CFS : Famine

- Le problème de famine ne se présente pas
  - Tout processus finira par utiliser l'UC
- Soit T1 une tâche qui n'a jamais obtenu l'UC tandis que T2 qui obtient tout le temps l'UC
  - T1 finira par avoir une valeur de vruntime vue que cette dernière ne s'est jamais incrémentée

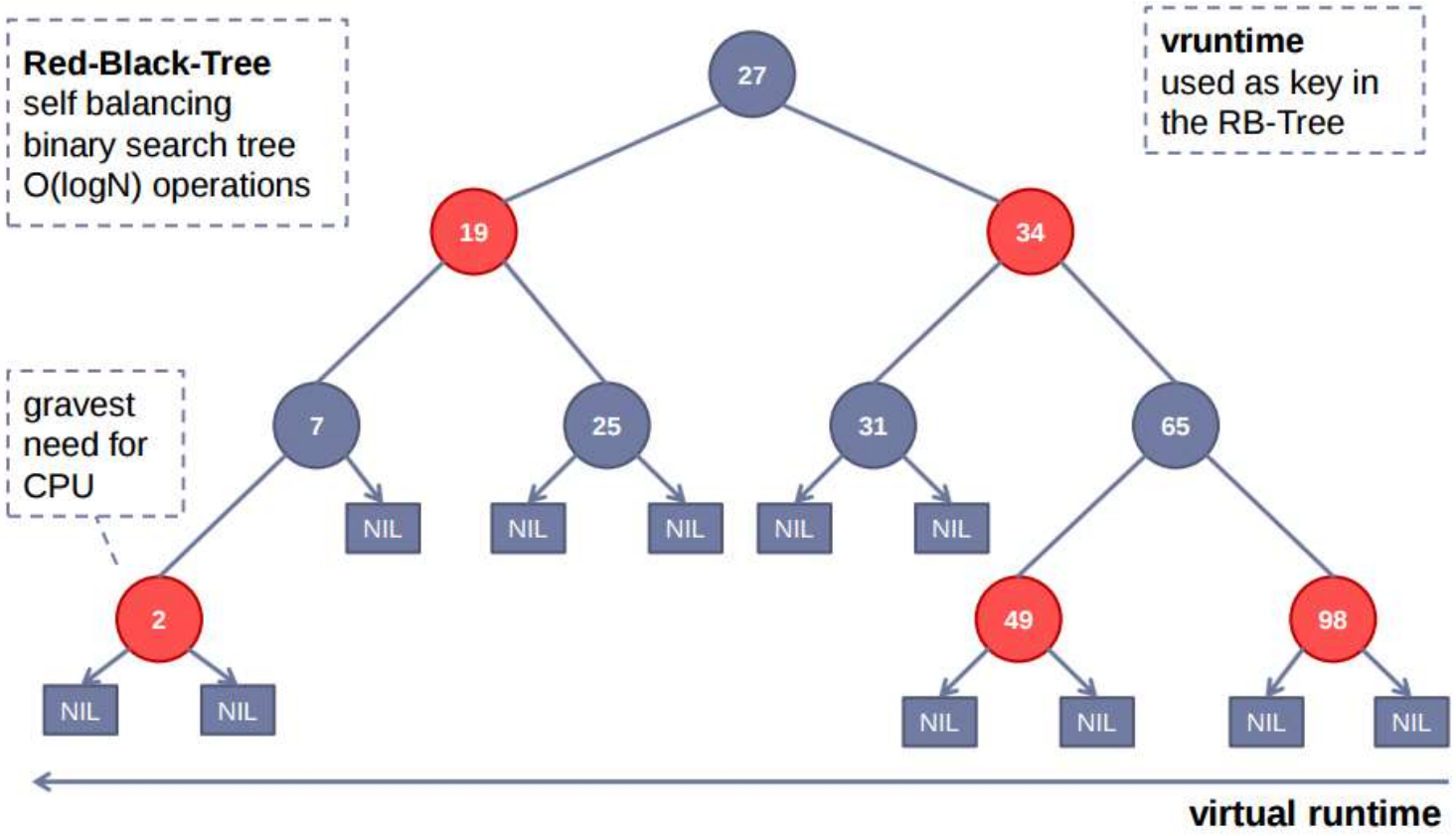
CFS : Sélection de Processus

- CFS sélectionne le processus avec le temps d'exécution virtuel le plus petit
- Evite d'avoir une file d'exécution (run queue) par niveau de priorité
- Quelle structure de données à utiliser pour représenter les tâches ?
  - Une seule file sera lente
  - Plusieurs files peuvent être intéressantes si on a un nombre relativement petit
  - Plusieurs valeurs de temps d'exécution virtuel

CFS : Arbre bicolore

- Un **arbre bicolore** ou **arbre rouge et noir** (**Red-Black Tree**) est un arbre binaire de recherche
  - Pour chaque nœud,
    - le sous arbre gauche contient uniquement des **clés plus petites** que la **clé du nœud**
    - le sous arbre droit contient des **clés plus grandes** que la **clé du nœud**
  - Le chemin le plus long possible d'une racine à une feuille (sa hauteur) ne peut être que deux fois plus long que le chemin le plus petit possible
    - Plus efficace que les arbres binaire de recherche
    - Les opérations sont de **O(log n)**

CFS : Arbre bicolore



https://www.ibm.com/developerworks/library/l-completely-fair-scheduler/

CFS : Exemple

- 5 tâches : T1, T2, T3, T4, et T5
- TL = 6
- Scheduling tick = 1ms

	nice	Wi	$W_i / \sum_{j=1}^{n} W_j$	Quantum
T1	-10	9548	?	?
T2	-5	3121	?	?
T3	0	1024	?	?
T4	5	335	?	?
T5	10	110	?	?
total		**?**	**?**	**?**

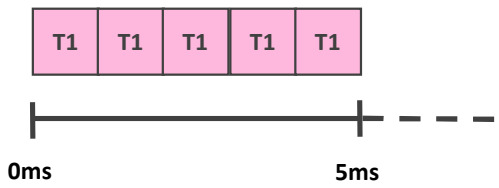
CFS : Exemple

- 5 tâches : T1, T2, T3, T4, et T5
- TL = 6
- Scheduling tick = 1ms

	nice	Wi	$W_i / \sum_{j=1}^{n} W_j$	Quantum
T1	-10	9548	0.6753	4.0518
T2	-5	3121	0.2208	1.3248
T3	0	1024	0.0724	0.4344
T4	5	335	0.0237	0.1422
T5	10	110	0.0078	0.0468
total		**14138**	**1.000**	**6**

CFS : Exemple

- Diagramme d'exécution

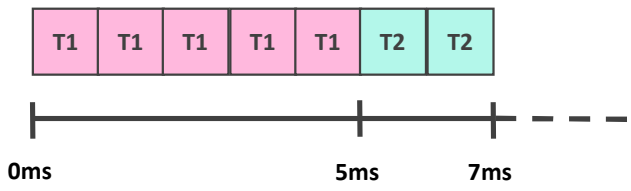


t = 0ms
vruntime1 = 0
vruntime2 = 0
vruntime3 = 0
vruntime4 = 0
vruntime5 = 0

t = 5ms
vruntime1 = 0.536
vruntime2 = 0
vruntime3 = 0
vruntime4 = 0
vruntime5 = 0

CFS : Exemple

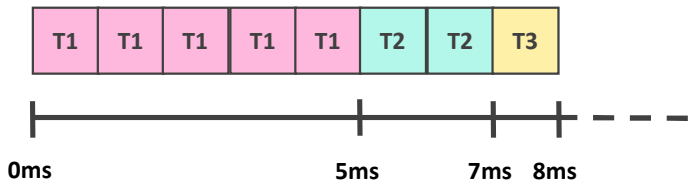
- Diagramme d'exécution



t = 7ms
vruntime1 = 0.536
vruntime2 = 0.6562
vruntime3 = 0
vruntime4 = 0
vruntime5 = 0

CFS : Exemple

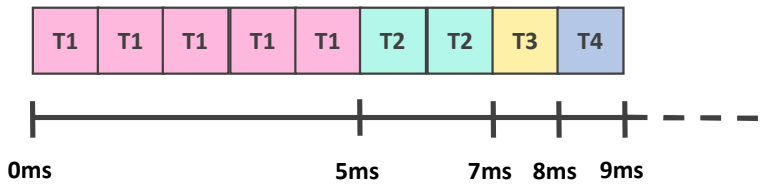
- Diagramme d'exécution



t = 8ms
vruntime1 = 0.536
vruntime2 = 0.6562
vruntime3 = 1
vruntime4 = 0
vruntime5 = 0

CFS : Exemple

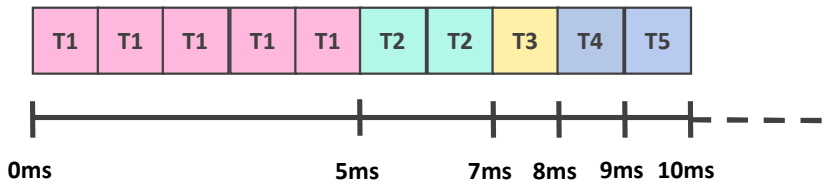
- Diagramme d'exécution



t = 9ms
vruntime1 = 0.536
vruntime2 = 0.6562
vruntime3 = 1
vruntime4 = 3.05
vruntime5 = 0

CFS : Exemple

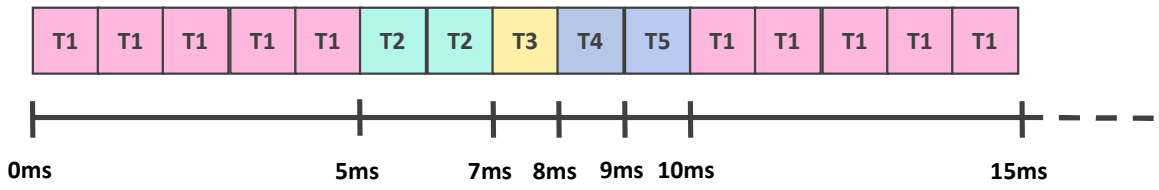
- Diagramme d'exécution



t = 10ms
vruntime1 = 0.536
vruntime2 = 0.6562
vruntime3 = 1
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

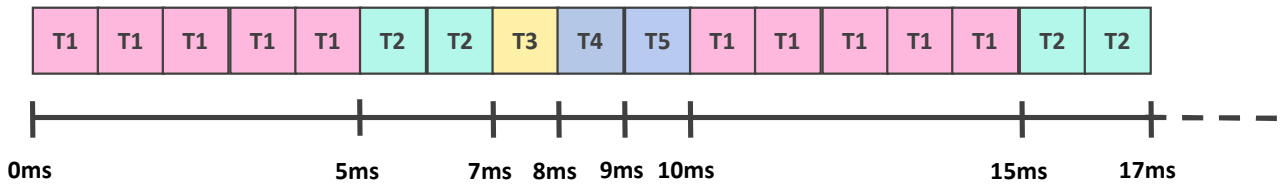
- Diagramme d'exécution



t = 15ms
vruntime1 = 1.072
vruntime2 = 0.6562
vruntime3 = 1
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

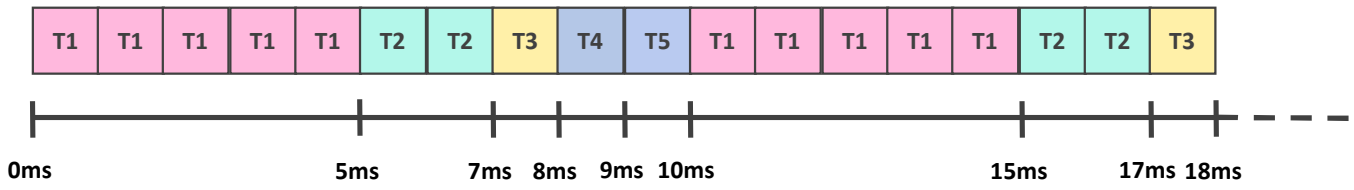
- Diagramme d'exécution



t = 17ms
vruntime1 = 1.072
vruntime2 = 1.3124
vruntime3 = 1
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

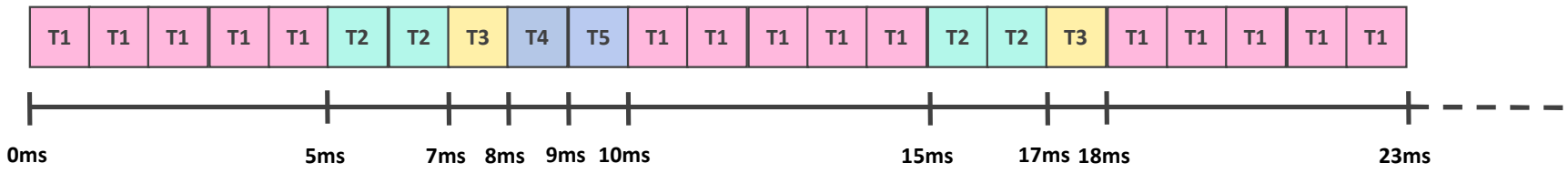
- Diagramme d'exécution



t = 18ms
vruntime1 = 1.072
vruntime2 = 1.3124
vruntime3 = 2
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

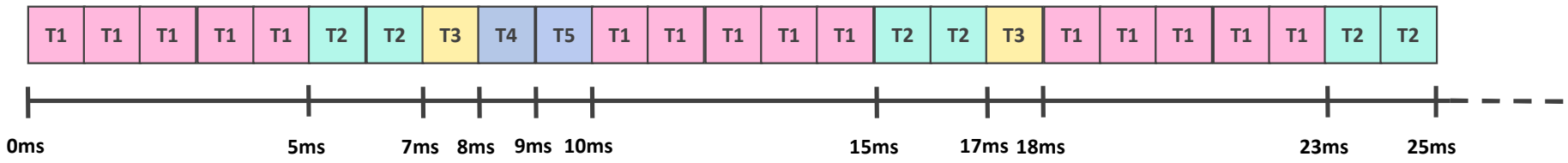
- Diagramme d'exécution



t = 23ms
vruntime1 = 1.608
vruntime2 = 1.3124
vruntime3 = 2
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

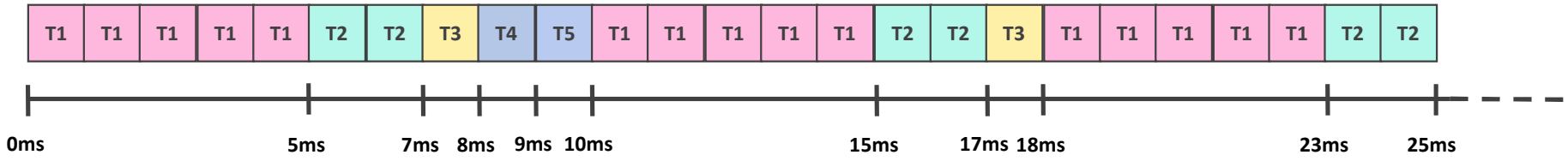
- Diagramme d'exécution

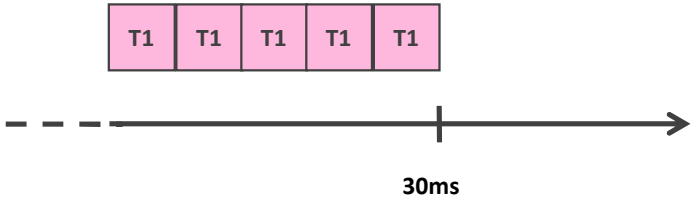


t = 25ms
vruntime1 = 1.608
vruntime2 = 1.9686
vruntime3 = 2
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

- Diagramme d'exécution

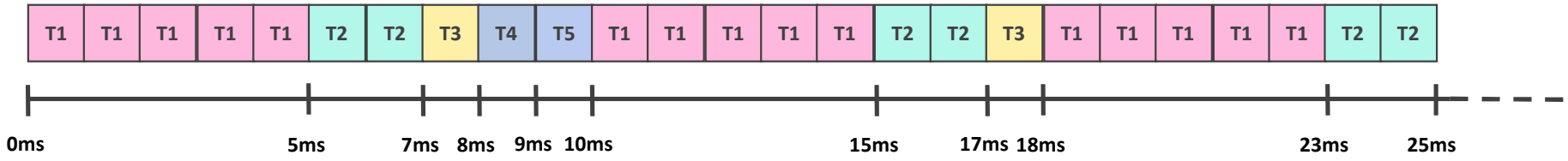


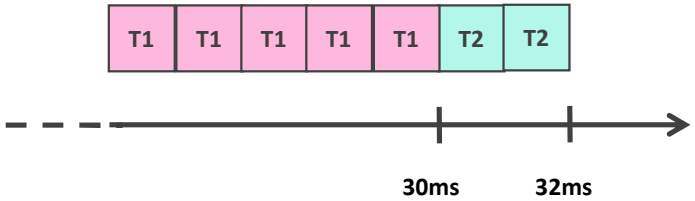


t = 30ms
vruntime1 = 2.144
vruntime2 = 1.9686
vruntime3 = 2
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

- Diagramme d'exécution

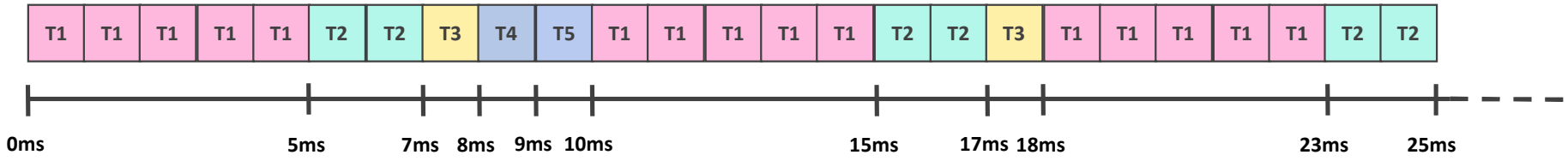


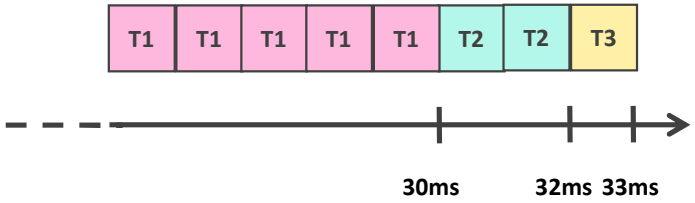


t = 32ms
vruntime1 = 2.144
vruntime2 = 2.6248
vruntime3 = 2
vruntime4 = 3.05
vruntime5 = 9.31

CFS : Exemple

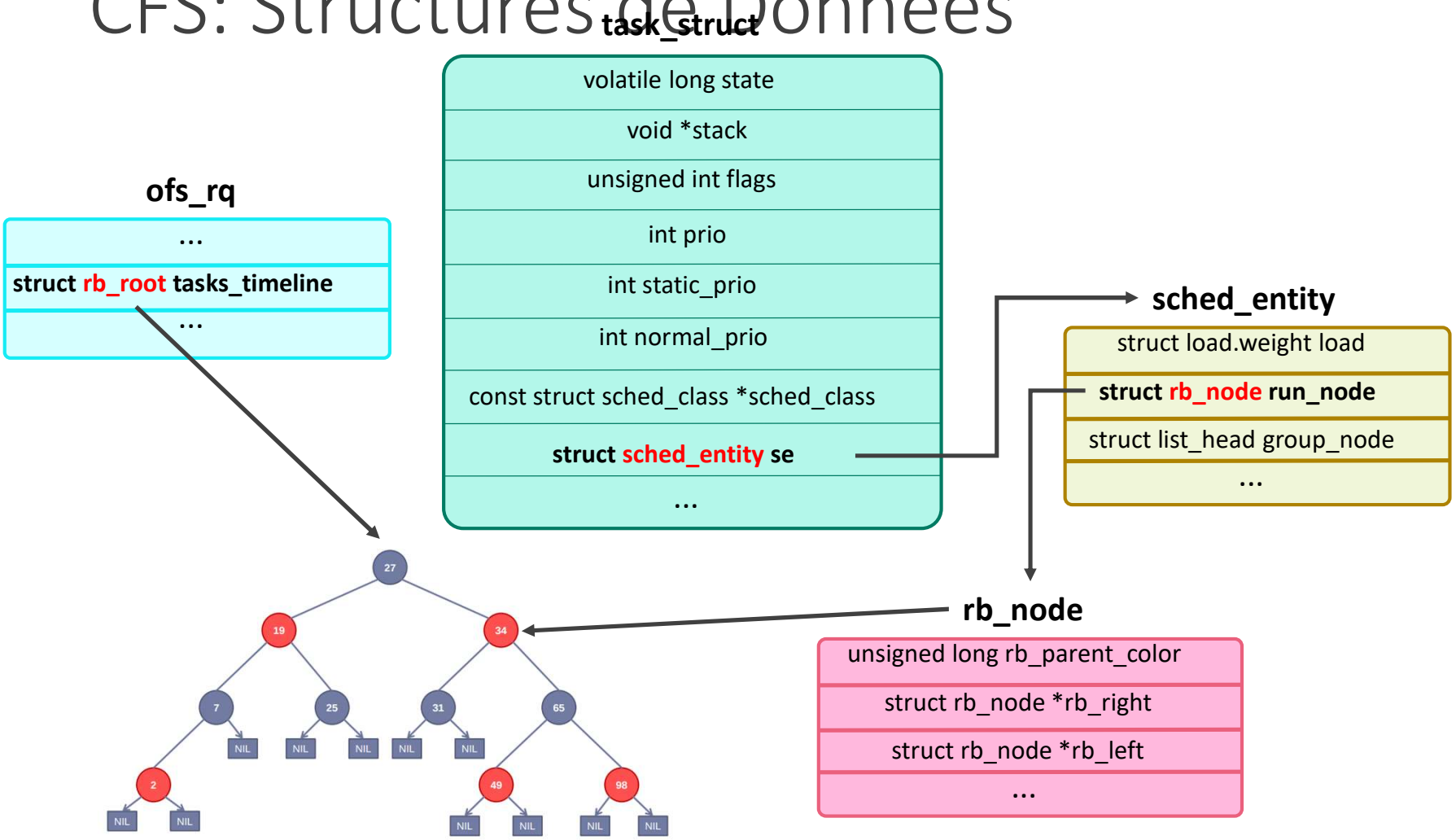
- Diagramme d'exécution





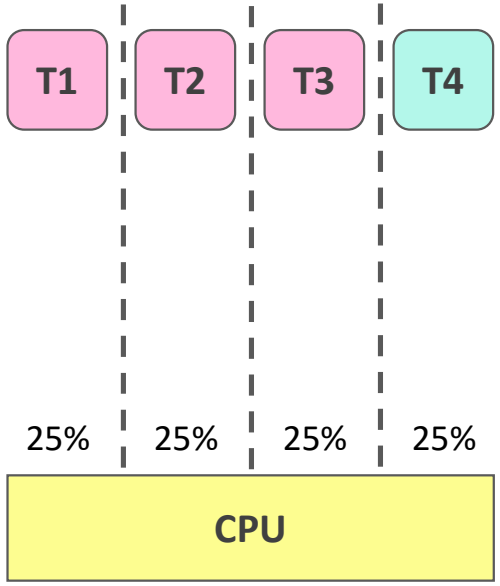
t = 33ms
vruntime1 = 2.144
vruntime2 = 2.6248
vruntime3 = 3
vruntime4 = 3.05
vruntime5 = 9.31

CFS: Structures de Données

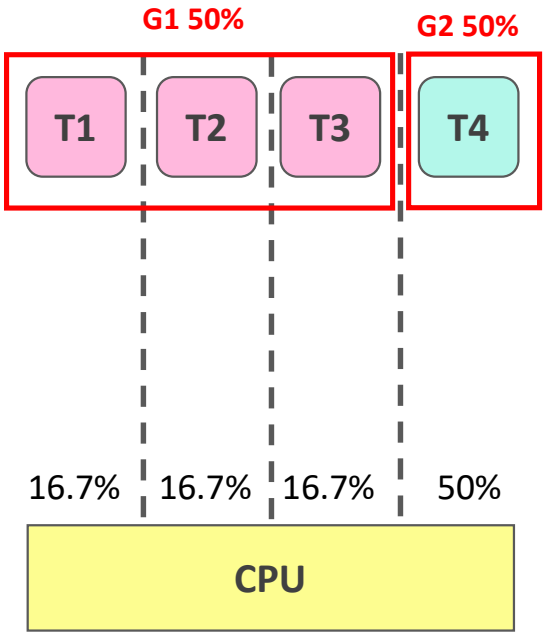


CFS: Scheduling de Groupes

Sans scheduling de groupes



Avec scheduling de groupes



Groupes de Contrôle (cgroups)

- Un ensemble de tâches (Threads)
- Des conteneurs Linux utilisés pour gérer l'allocation de temps CPU pour tous les threads dans un conteneur
- Tous les threads créés dans une App appartiennent à un des cgroups
- Android définis plusieurs cgroups. Les plus importants sont:
  - Foreground Group
  - Background Group

Groupes de Contrôle (cgroups)

- Threads dans des cgroups différents sont alloués différentes tranches de temps CPU
- Threads dans Foreground Group sont alloués beaucoup plus de temps CPU par rapport aux threads dans Background Group
  - Les threads dans Background Group ne peuvent pas avoir plus que **5-10%** de temps CPU **tous ensemble**
    - **init.rc**
  - Assurer que les applications visibles sur l'écran reçoivent plus de temps CPU par rapport aux applications non visibles sur l'écran
    - La visibilité sur l'écran est reliée au niveau d'importance du processus

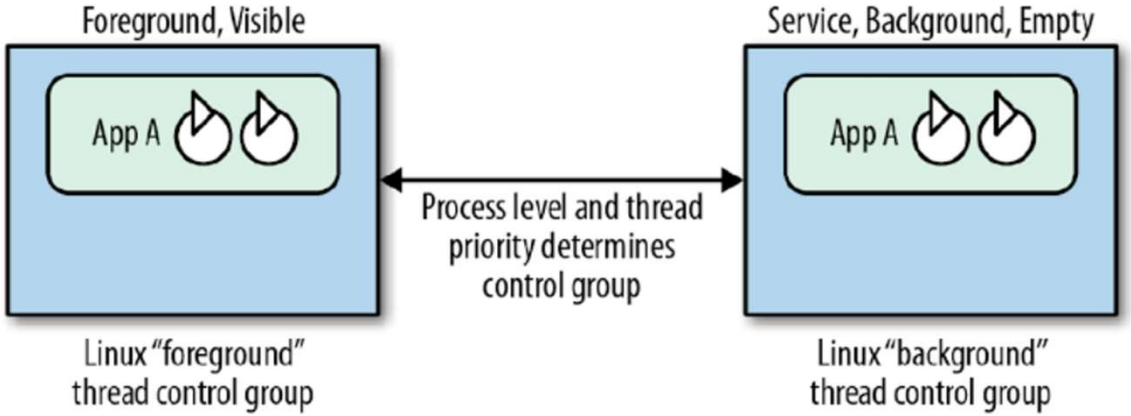
Groupes de Contrôle (cgroups)

init.rc

```
mkdir /dev/cpuctl/apps/bg_non_interactive
chown system system /dev/cpuctl/apps/bg_non_interactive/tasks
chmod 0666 /dev/cpuctl/apps/bg_non_interactive/tasks
# 5.0 %
write /dev/cpuctl/apps/bg_non_interactive/cpu.shares 52
```

(52/1024)*100 ≈ 5%

Groupes de Contrôle (cgroups)



Source : Efficient Android Threading: Asynchronous Processing Techniques for Android Applications

```
$ adb shell ps –P | grep u0_a72
u0_a72   4257   144   320304   34504   fg   ffffffff   00000000   S   com.eat
```

```
$ adb shell ps –P | grep u0_a72
u0_a72   4257   144   318700   32164   bg   ffffffff   00000000   S   com.eat
```